



Univerzitet „Džemal Bijedić“ u Mostaru

Fakultet Informacijskih Tehnologija

Predmet: Umjetna inteligencija

HealthPredict – AI agent za procjenu rizika od srčanih bolesti

Dokumentacija

Profesor:
prof.dr.sc. Nina Bijedić

Student:
Nejra Muminović, IB220043

Mostar, 2025

Svrha i cilj projekta

Cilj projekta HealthPredict je razvoj AI agenta koji omogućava procjenu rizika od srčanih bolesti na osnovu medicinskih i demografskih podataka pacijenata. Sistem je namijenjen kao pomoćni alat za brzu i informativnu procjenu zdravstvenog rizika, a ne kao zamjena za stručnu medicinsku dijagnozu.

Projekt je inspiriran činjenicom da su kardiovaskularne bolesti jedan od vodećih uzroka smrtnosti u svijetu, te da rana procjena rizika može imati ključnu ulogu u prevenciji i pravovremenom liječenju. Korištenjem historijskih medicinskih podataka i algoritama mašinskog učenja, HealthPredict omogućava korisnicima da dobiju uvid u vjerovatnoću postojanja srčanih oboljenja.

Svrha projekta je izrada web aplikacije koja koristi algoritme mašinskog učenja za analizu faktora kao što su starost, spol, tip angine, krvni pritisak, holesterol, EKG rezultati i drugi relevantni parametri, te na osnovu njih predviđa rizik od srčanih bolesti.

Tehnologije

Za realizaciju AI agenta HealthPredict korištene su sljedeće tehnologije:

- Python – korišten za backend logiku, obradu podataka, treniranje modela i predikciju.
- Flask – korišten za izradu REST API-ja i komunikaciju između frontenda i backend-a.
- Random Forest Classifier – algoritam mašinskog učenja korišten za klasifikaciju rizika od srčanih bolesti.
- Logistic Regression – korišten kao dodatni model tokom treniranja i evaluacije.
- JavaScript – korišten za izradu frontend dijela aplikacije i komunikaciju s API-jem.
- HTML & CSS – korišteni za izradu korisničkog interfejsa.
- Pandas – korišten za obradu i manipulaciju medicinskih podataka.
- Scikit-learn – biblioteka korištena za treniranje, skaliranje i evaluaciju ML modela.
- CSV dataset (Heart Disease Dataset) – korišten kao izvor podataka za treniranje modela.

Zašto Random Forest Classifier?

Random Forest Classifier je odabran zbog svoje sposobnosti da:

- efikasno radi s većim brojem karakteristika,
- bude otporan na šum u podacima,
- pruži dobru tačnost bez potrebe za kompleksnim podešavanjem,
- omogući uvid u važnost pojedinih karakteristika koje utiču na predikciju.

Zahvaljujući ovim osobinama, Random Forest je pogodan za medicinske podatke koji često sadrže varijacije i nelinearne odnose.

Funkcionalnosti

AI agent HealthPredict nudi sljedeće funkcionalnosti:

- Predikcija rizika od srčanih bolesti
Korisnik unosi medicinske podatke (starost, spol, tip bola u prsima, krvni pritisak, kolesterol, EKG rezultat, puls, itd.), a sistem vraća procjenu rizika (nizak ili visok rizik) zajedno sa vjerovatnoćom.
- Prikaz najvažnijih faktora
Sistem prikazuje koje karakteristike su najviše utjecale na odluku modela.
- Dodavanje novih pacijenata
Korisnici mogu dodavati nove podatke o pacijentima u bazu podataka.
- Ponovno treniranje modela (Retraining)
Nakon dodavanja novih podataka, model se može ponovno trenirati kako bi se poboljšala tačnost i prilagodio novim informacijama.
- Višejezična podrška (Bosanski / Engleski)
Korisnički interfejs omogućava promjenu jezika aplikacije.

Kod

Učitavanje podataka

Podaci se učitavaju iz CSV datoteke korištenjem biblioteke Pandas. Datoteka sadrži medicinske i demografske podatke pacijenata koji se koriste za procjenu rizika od srčanih bolesti.

```
6 def load_data(path="data/heart.csv"):
7     df = pd.read_csv(path)
8     return df
```

Funkcija `load_data` omogućava centralizovano i jednostavno učitavanje podataka iz fajla `heart.csv`, koji predstavlja zajednički izvor podataka za treniranje modela, dodavanje novih pacijenata i agentnu obradu.

Učitani skup podataka uključuje attribute kao što su starost, spol, krvni pritisak, kolesterol, tip bola u prsima, EKG nalaz i druge klinički relevantne varijable. Ovi podaci služe kao ulaz u daljnje faze sistema, uključujući predobradu, treniranje modela i donošenje odluka od strane inteligentnog agenta.

Procesiranje podataka

Podaci se čiste i pripremaju za treniranje modela:

- uklanjaju se nedostajuće vrijednosti,
- kreira se ciljna varijabla (0 – nema bolesti, 1 – postoji rizik),
- vrši se enkodiranje kategorijskih varijabli,
- numeričke vrijednosti se skaliraju korištenjem StandardScaler.

```
10 def preprocess(df):
11     df = df.dropna(how="all")
12
13     if "target" not in df.columns:
14         df["target"] = df["num"].apply(lambda x: 1 if x > 0 else 0)
15
16     y = df["target"]
17     X = df.drop(columns=["target", "num", "id", "dataset"], errors="ignore")
18
19     numeric_cols = X.select_dtypes(include=["int64", "float64"]).columns
20     cat_cols = X.select_dtypes(include=["object"]).columns
21
22     imputer = SimpleImputer(strategy="mean")
23     X_numeric = pd.DataFrame(
24         imputer.fit_transform(X[numeric_cols]),
25         columns=numeric_cols
26     )
27
28     X_cat = pd.get_dummies(X[cat_cols], drop_first=True)
29
30     X_processed = pd.concat([X_numeric, X_cat], axis=1)
31
32     scaler = StandardScaler()
33     X_scaled = scaler.fit_transform(X_processed)
34
35     return X_scaled, y, scaler, X_processed.columns.tolist()
```

Podaci se prije treniranja modela prolaze kroz fazu procesiranja. Uklanjaju se redovi sa nedostajućim vrijednostima, zatim se kreira ciljna varijabla gdje vrijednost 1 označava prisustvo rizika od srčanih bolesti, dok 0 označava odsustvo rizika. Kategorijske varijable se enkodiraju korištenjem binarnog mapiranja i one-hot enkodiranja, dok se numeričke vrijednosti skaliraju pomoću StandardScaler metode radi poboljšanja performansi modela.

Treniranje modela

Model se trenira korištenjem Random Forest Classifier algoritma. Tokom procesa treniranja čuvaju se sljedeće komponente:

- istrenirani model,
- scaler korišten za normalizaciju numeričkih podataka,
- lista feature-a koji su korišteni prilikom treniranja.

Ove komponente se kasnije koriste tokom predikcije kako bi se osiguralo da se novi ulazni podaci obrađuju na isti način kao i podaci korišteni za treniranje modela.

```
11  def main():
12      df = load_data()
13      X, y, scaler, features = preprocess(df)
14
15      X_train, X_test, y_train, y_test = train_test_split(
16          X, y, test_size=0.2, random_state=42
17      )
18
19      model = RandomForestClassifier(
20          n_estimators=200,
21          random_state=42
22      )
23      model.fit(X_train, y_train)
24
25      score = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
26      print("ROC AUC:", score)
27
28      os.makedirs("models", exist_ok=True)
29      joblib.dump(
30          {"model": model, "scaler": scaler, "features": features},
31          MODEL_PATH
32      )
33      print("Model saved.")
34
```

Proces treniranja započinje podjelom procesiranih podataka na trening i test skup, pri čemu se 80% podataka koristi za treniranje, a 20% za evaluaciju modela. Random Forest model se inicijalizira sa unaprijed definisanim parametrima, uključujući `n_estimators = 200`, dok se parametar `random_state = 42` koristi radi osiguravanja ponovljivosti rezultata.

Nakon treniranja modela, njegove performanse se evaluiraju pomoću ROC AUC metrike, koja je posebno pogodna za medicinske klasifikacijske probleme jer mjeri sposobnost modela da razlikuje rizične i nerizične slučajeve nezavisno od izabranog praga odlučivanja.

Po završetku treniranja i evaluacije, istrenirani model zajedno sa scalerom i listom feature-a trajno se sprema na disk. Ovi podaci se kasnije učitavaju unutar agentne logike aplikacije i koriste za procjenu rizika od srčanih bolesti kod novih pacijenata.

Predikcija rizika srčanih bolesti

Predikcija rizika srčanih bolesti realizirana je kroz agentnu arhitekturu koja odvaja prijem korisničkog zahtjeva od njegove obrade. Sistem se sastoji od API sloja i autonomnog agenta koji se izvršava u pozadini.

Korisnik putem web aplikacije unosi medicinske i demografske podatke pacijenta, nakon čega frontend šalje zahtjev prema backend API-ju. API endpoint ne izvršava predikciju direktno, već zaprimljene podatke smješta u red čekanja (queue) i korisniku vraća jedinstveni identifikator zahtjeva.

Autonomni agent (HealthRiskAgent) u pozadini periodično provjerava red čekanja i, kada preuzme novi zahtjev, pokreće kompletan Sense–Think–Act ciklus. U fazi Sense agent preuzima podatke pacijenta, u fazi Think koristi istrenirani Random Forest model za izračunavanje vjerovatnoće rizika od srčanih bolesti, dok u fazi Act donosi konačnu kliničku odluku.

```
11 def agent_loop():
12     print("🧠 HealthAgent runner started")
13
14     while True:
15         item = get_request()
16         if item is None:
17             time.sleep(0.5)
18             continue
19
20         request_id, data = item
21
22         percept = agent.sense(data)
23         risk = agent.think(percept)
24         decision = agent.act(risk, data)
25         explanation = agent.explain(data, decision)
26
27         save_result(request_id, {
28             "risk": risk,
29             "decision": decision,
30             "explanation": explanation
31         })
```

Konačna odluka agenta ne zavisi isključivo od statističke vjerovatnoće modela, već i od medicinskih pravila, kao što su povišen krvni pritisak, visok kolesterol, prisustvo angine pri naporu i broj zahvaćenih koronarnih arterija.

Na osnovu kombinacije ovih faktora, agent klasificira pacijenta u jednu od sljedećih kategorija:

- Nizak rizik (LOW_RISK),
- Potrebna dodatna procjena (REVIEW),
- Visok rizik (HIGH_RISK).

Rezultat predikcije uključuje:

- procijenjenu vjerovatnoću rizika (izraženu u procentima),
- konačnu kliničku odluku agenta,
- tekstualno medicinsko objašnjenje koje navodi glavne faktore rizika.

Predikcija se nakon obrade pohranjuje u internu strukturu rezultata, a frontend periodičnim upitima dohvaća gotov rezultat i prikazuje ga korisniku na jasan i vizuelno prilagođen način.

Dodavanje novih podataka

Dodavanje novih podataka o pacijentima implementirano je putem backend API endpointa /add. Ova funkcionalnost omogućava unos novih medicinskih i demografskih podataka, koji se pohranjuju u postojeći skup podataka i koriste za buduće ponovno treniranje modela.

```
33 @app.route("/add", methods=["POST"])
34 def add_data():
35     try:
36         data = request.json
37
38         required_fields = [
39             "age", "sex", "cp", "trestbps", "chol",
40             "fbs", "restecg", "thalch", "exang",
41             "oldpeak", "slope", "ca", "thal"
42         ]
43
44         for field in required_fields:
45             if field not in data or data[field] in ["", None]:
46                 return jsonify({
47                     "status": "error",
48                     "message_bs": "Sva polja moraju biti popunjena.",
49                     "message_en": "All fields must be filled."
50                 }), 400
51
52         df = load_data()
53
54         data.setdefault("id", len(df) + 1)
55         data.setdefault("dataset", "new")
56         data.setdefault("num", 0)
57
58         df = pd.concat([df, pd.DataFrame([data])], ignore_index=True)
59         df.to_csv("data/heart.csv", index=False)
60
61         return jsonify({"status": "success"})
62
63     except Exception as e:
64         return jsonify({
65             "status": "error",
66             "message": str(e)
67         }), 500
```

Prilikom zaprimanja zahtjeva, backend prima podatke u JSON formatu i provjerava da li su sva obavezna polja popunjena. Obavezna polja uključuju medicinske i kliničke

atribute kao što su starost, spol, tip angine, krvni pritisak, kolesterol, šećer u krvi, EKG nalaz, maksimalni puls, angina pri naporu, vrijednost oldpeak, nagib ST segmenta, broj zahvaćenih krvnih sudova i thal test. Ukoliko neko od obaveznih polja nije prisutno ili je prazno, zahtjev se odbija i korisniku se vraća odgovarajuća poruka o grešci.

Nakon uspješne validacije, sistem učitava postojeće podatke iz CSV datoteke i automatski dodaje interne kolone:

- id – jedinstveni identifikator zapisa,
- dataset – oznaka da se radi o novododanom zapisu,
- num – početna vrijednost ciljne varijable.

Novi zapis se zatim dodaje u postojeći DataFrame i trajno pohranjuje u CSV datoteku. Na ovaj način, sistem omogućava postepeno proširivanje skupa podataka, dok se novododani pacijenti kasnije koriste u fazi učenja kroz ponovno treniranje modela. U slučaju greške tokom obrade zahtjeva, backend vraća JSON odgovor sa opisom problema.

Ponovno treniranje modela

Sistem omogućava ponovno treniranje modela korištenjem proširenog skupa podataka, čime se osigurava da agent kontinuirano unapređuje kvalitet svojih budućih odluka. Ova funkcionalnost predstavlja Learn fazu agentnog ciklusa, jer se model prilagođava novododanim podacima o pacijentima.

Ponovno treniranje se pokreće putem backend API endpointa /retrain. Kada korisnik pošalje POST zahtjev na ovaj endpoint, backend inicira proces treniranja novog modela pozivom zasebne skripte train_model.py. Ova skripta učitava ažurirani dataset, vrši procesiranje podataka i trenira novi Random Forest Classifier model.

Tokom treniranja, podaci se dijele na trening i test skup, nakon čega se model uči na trening podacima. Performanse modela se evaluiraju korištenjem ROC AUC metrike, čime se dobija uvid u kvalitet predikcija. Nakon završetka treniranja, istrenirani model, zajedno sa scalerom i listom feature-a, trajno se sprema na disk kako bi bio dostupan za buduće predikcije.

Nakon uspješnog završetka procesa, backend vraća JSON odgovor koji korisniku potvrđuje da je model uspješno retreniran. U slučaju greške tokom procesa, sistem vraća odgovarajuću poruku o grešci. Ovakav pristup omogućava jednostavno i kontrolisano ponovno treniranje modela bez potrebe za restartovanjem aplikacije.

- Endpoint za ponovno treniranje (api.py)

```

70 @app.route("/retrain", methods=["POST"])
71 def retrain_model():
72     try:
73         import os
74         os.system("python train_model.py")
75
76         return jsonify({
77             "status": "success",
78             "message_bs": "Model je uspješno retreniran.",
79             "message_en": "Model retraining completed successfully."
80         })
81     except Exception as e:
82         return jsonify({
83             "status": "error",
84             "message_bs": "Greška prilikom retreniranja.",
85             "message_en": "Error during retraining."
86         }), 500
87

```

- Skripta za treniranje modela (train_model.py)

```

11 def main():
12     df = load_data()
13     X, y, scaler, features = preprocess(df)
14
15     X_train, X_test, y_train, y_test = train_test_split(
16         X, y, test_size=0.2, random_state=42
17     )
18
19     model = RandomForestClassifier(
20         n_estimators=200,
21         random_state=42
22     )
23     model.fit(X_train, y_train)
24
25     score = roc_auc_score(y_test, model.predict_proba(X_test)[: , 1])
26     print("ROC AUC:", score)
27
28     os.makedirs("models", exist_ok=True)
29     joblib.dump(
30         {"model": model, "scaler": scaler, "features": features},
31         MODEL_PATH
32     )
33     print("Model saved.")

```

Agentni ciklus (Sense – Think – Act – Learn)

Sistem je implementiran kao autonomni softverski agent koji funkcioniše nezavisno od API sloja. Agent ne reaguje direktno na HTTP zahtjeve, već radi u pozadini kroz kontinuirani ciklus obrade.

Agentni ciklus se sastoji od sljedećih faza:

- **Sense** – Agent preuzima nove zahtjeve iz reda čekanja (queue), koji predstavlja njegovo okruženje.

```
12 def add_request(data):
13     request_id = str(uuid.uuid4())
14     with queue_lock:
15         pending_requests.append((request_id, data))
16     return request_id
```

(agent_runtime.py)

```
15 item = get_request()
```

(agent_runner.py)

- **Think** – Agent koristi istrenirani ML model za izračun vjerovatnoće rizika od srčanih bolesti.

```
10 def think(self, data):
11     import pandas as pd
12     X = pd.DataFrame([data])
13     X = X.reindex(columns=self.features, fill_value=0)
14     X_scaled = self.scaler.transform(X)
15     return float(self.model.predict_proba(X_scaled)[0][1])
```

(health_agent.py)

- **Act** – Na osnovu izračunate vjerovatnoće i medicinskih pravila, agent donosi diskretnu odluku (nizak ili visok rizik).

```
17 def act(self, risk, data):
18     critical = 0
19
20     if data.get("age", 0) >= 60: critical += 1
21     if data.get("cp") == "asymptomatic": critical += 1
22     if data.get("trestbps", 0) >= 150: critical += 1
23     if data.get("chol", 0) >= 300: critical += 1
24     if data.get("exang") in [1, True, "1"]: critical += 1
25     if data.get("oldpeak", 0) >= 2.5: critical += 1
26     if data.get("slope") == "downsloping": critical += 1
27     if data.get("ca", 0) >= 2: critical += 1
28     if data.get("thal") == "reversable defect": critical += 1
29
30     if critical >= 3:
31         return "HIGH_RISK"
32     if risk >= 0.7:
33         return "HIGH_RISK"
34     if risk <= 0.3:
35         return "LOW_RISK"
36     return "REVIEW"
```

(health_agent.py)

- **Learn** – Sistem omogućava ponovno treniranje modela na osnovu novih podataka, čime agent poboljšava svoje buduće odluke.

```

70 @app.route("/retrain", methods=["POST"])
71 def retrain_model():
72     try:
73         import os
74         os.system("python train_model.py")
75
76         return jsonify({
77             "status": "success",
78             "message_bs": "Model je uspješno retreniran.",
79             "message_en": "Model retraining completed successfully."
80         })
81     except Exception as e:
82         return jsonify({
83             "status": "error",
84             "message_bs": "Greška prilikom retreniranja.",
85             "message_en": "Error during retraining."
86         }), 500
87

```

(api.py)

```

19 model = RandomForestClassifier(
20     n_estimators=200,
21     random_state=42
22 )
23 model.fit(X_train, y_train)

```

(train_model.py)

Na ovaj način je jasno razdvojena logika odlučivanja agenta od web API sloja, što zadovoljava principe agentne arhitekture.

API kao komunikacijski sloj

API sloj ima isključivo ulogu komunikacije s korisnikom i ne sadrži logiku odlučivanja. Njegove odgovornosti su:

- prijem korisničkih zahtjeva,
- validacija ulaznih podataka,
- dodavanje zahtjeva u red čekanja,
- vraćanje statusa obrade i rezultata.

Agent nikada nije direktno pozivan iz API endpointa, već samostalno obrađuje zahtjeve u pozadini.

Okruženje agenta (Queue)

Uloga okruženja agenta implementirana je pomoću reda čekanja (`pending_requests`). API endpoint `/predict` dodaje zahtjev u red, dok agent u pozadinskoj petlji provjerava da li postoje novi zahtjevi za obradu.

Ovakav pristup omogućava:

- asinhronu obradu zahtjeva,
- nezavisnost agenta od korisničkih poziva,
- realističnu simulaciju rada autonomnog sistema.

Kombinacija statističkog i kliničkog odlučivanja

Odluka agenta ne zavisi isključivo od izlaza ML modela. Pored statističke vjerovatnoće, agent koristi i skup medicinskih pravila (rule-based sistem) koji uključuje klinički značajne faktore kao što su:

- starija životna dob,
- visoki krvni pritisak,
- visok holesterol,
- angina pri naporu,
- ST depresija,
- zahvaćenost više koronarnih arterija.

Ako je prisutno više kritičnih faktora, agent može donijeti odluku o visokom riziku čak i kada je statistička vjerovatnoća niža. Time se postiže realističnije i sigurnije ponašanje sistema, bliže kliničkoj praksi.

Korisnički interfejs

Interfejs za predikciju rizika od srčanih bolesti:

HealthPredict

Jezik: Bosanski

Predikcija bolesti srca

Starost

40-70

Spol

Muški

Tip angine

tipična angina

Krvni pritisak u mirovanju

100-130

Holesterol

150-300

Šećer u krvi na prazan stomak

Ne

EKG u mirovanju

normalno

Maksimalni puls

60-120

Angina izazvana vježbom

Ne

Oldpeak

0-6

Nagib ST segmenta

ravan

Oldpeak

0-6

Nagib ST segmenta

ravan

CA

0-3

Thal

normalno

Predvidi rizik

Ovdje će biti prikazan rezultat procjene rizika za srčane bolesti.

Interfejs za dodavanje novih podataka:

HealthPredict

Jezik: Bosanski

Dodaj novog pacijenta

Starost

40-70

Spol

Odaberi

Tip angine

Odaberi

Krvni pritisak u mirovanju

100-130

Holesterol

150-300

Šećer u krvi na prazan stomak

Odaberi

EKG u mirovanju

Odaberi

Maksimalni puls

60-120

Angina izazvana vježbom

Odaberi

Oldpeak

0-6

Nagib ST segmenta

Odaberi

CA

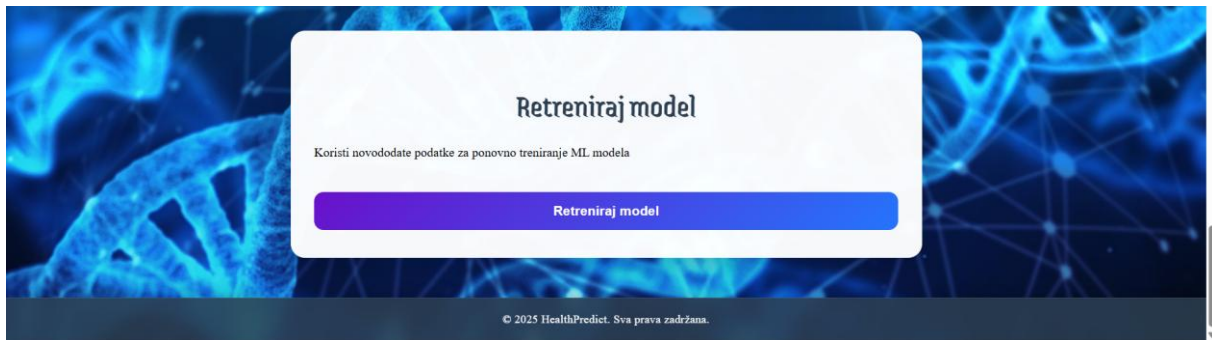
0-3

Thal

Odaberi

Dodaj pacijenta

Interfejs za ponovno treniranje modela:



Zaključak

Aplikacija HealthPredict je razvijena kao autonomni softverski agent za procjenu rizika od srčanih bolesti, koji objedinjuje mašinsko učenje i domenska medicinska pravila u jedinstven sistem odlučivanja. Za razliku od klasičnih ML web aplikacija, HealthPredict ne reaguje direktno na korisničke zahtjeve, već funkcioniše kroz agentni ciklus Sense–Think–Act–Learn, čime se postiže jasna razlika između API sloja i logike odlučivanja.

U fazi Think, agent koristi istrenirani Random Forest model za izračun statističke vjerovatnoće rizika, dok se u fazi Act donosi diskretna klinička odluka na osnovu kombinacije rezultata modela i medicinskih pravila (npr. starost pacijenta, krvni pritisak, holesterol i drugi klinički faktori). Ovakav hibridni pristup omogućava realističnije i sigurnije procjene rizika, bliže stvarnoj kliničkoj praksi nego sama statistička inferenca.

Sistem je dodatno proširen mogućnošću dodavanja novih podataka o pacijentima i ponovnog treniranja modela, čime se omogućava kontinuirano unapređenje performansi i prilagođavanje modela novim podacima. Time je realizirana i Learn faza agentnog ciklusa, što sistem čini adaptivnim i dugoročno održivim.

Korisnički interfejs aplikacije je jednostavan i intuitivan, uz podršku za više jezika, vizuelni prikaz rizika i jasno objašnjenje odluka koje agent donosi. Na taj način HealthPredict ne pruža samo numeričku procjenu rizika, već i transparentno objašnjenje razloga koji su doveli do određene odluke, što povećava povjerenje korisnika u sistem.

Sve navedeno čini HealthPredict primjerom prave agentne AI aplikacije, koja jasno razdvaja percepciju, odlučivanje i akciju, te demonstrira značajan potencijal primjene autonomnih agenata i vještačke inteligencije u oblasti zdravstva, posebno u kontekstu rane procjene kardiovaskularnih rizika i podrške donošenju medicinskih odluka.